

Query 1: From 1995 Onwards, Find Movies with the Comedy Genre

Solution:

1. **Filter Movies Dataset:** Load the movies.csv file and filter movies released after 1995.
2. **Extract Comedy Movies:** Identify movies tagged as "Comedy" from the genre list.
3. **Group by Year:** Group the results by year to collect all Comedy movies for each year.

```
[1]: from pyspark import SparkContext
sc = SparkContext("local", "test")
print(sc.version)

24/12/09 16:51:14 WARN Utils: Your hostname, Anandaa-Mac.local resolves to a loopback address: 127.0.0.1; using 100.69.37.249 instead (on 1
interface en0)
24/12/09 16:51:14 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging for this class or its sub-classes, use the conf log.level setting.
24/12/09 16:51:14 WARN NativeCodeLoader: Unable to load native-udf library for your platform... using builtin-java classes where applicab
le
24/12/09 16:51:15 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.

[2]: rdd = sc.parallelize([1, 2, 3, 4, 5])
print(rdd.collect()) # Should print [1, 2, 3, 4, 5]

[1, 2, 3, 4, 5]

[3]: from io import StringIO
import csv

[4]: def split_complex(x):
    return list(csv.reader(StringIO(x), delimiter=','))[0]

[5]: def split_genre(x):
    return list(csv.reader(StringIO(x), delimiter=','))[6]

[6]: def get_year(data):
    return data.split(",")[1].split(",")[0]

[7]: def get_title(data):
    return data.split(",")[0]

[8]: moviesRDD = sc.textFile("movies.csv")
header = moviesRDD.first()
moviesRDD1 = moviesRDD.filter(lambda line: line != header)

[9]: moviesRDD2 = moviesRDD1.map(lambda x: (int(get_year(split_complex(x))), get_title(split_complex(x)), split_complex(x)[0], split_genre(

[10]: moviesAfter1995RDD = moviesRDD2.filter(lambda x: x[0] >= 1995)
comedyMoviesAfter1995 = moviesAfter1995RDD.map(lambda x: (x[0], x[1], x[2]), x[3])).flatMapValues(lambda x: x).filter(lambda x: x[1] == 'Co

[11]: Q1ResultRDD = comedyMoviesAfter1995.groupByKey().mapValues(list)
Q1ResultRDD.take(1)

[12]: [(1995,
['Toy Story ',
'Grumpier Old Men ',
'Waiting to Exhale ',
'Father of the Bride Part II ',
'Sabrina ',
'American President, The ',
'Spiral: Dead and Living II ',
'Four Rooms ',
'Ace Ventura: When Nature Calls ',
'Money Train ',
'Get Shorty ',
'It Takes Two ',
'Clueless ',
'In the Line of Fire ',
'Mighty Aphrodite ',
'Big Screen, The ',
'Friday ',
'Kicking and Screaming ',
'Vampire in Brooklyn ',
'In the Black Webmaster ',
'Trash Bag, Small Little ',
'Brotherhood: The ',
'Bad Boys ',
'Batman Forever ',
'Blind in the Face ',
'Canadian Bacon ',
'Queen Generation, The ',
'Jeffrey ',
'Jury Duty ',
'Living in Oblivion ',
'Mallrats ',
'Nine Months ',
'Party Girl ',
'Reckless ',
'Snacks ',
'Something to Talk About ',
'To Wong Foo, Thanks for Everything! Julie Newmar ',
'Unstrung Heroes ',
'Boothhugged ',
'Billi Melendez ',
'Boys on the Side ',
'Dean Cain Defiance ',
'Destiny Turns on the Radio ',
'French Kiss ',
'Forget Paris ',
'Guffy Movie, A ',
'Guppy ',
'Jerky Boys, The ',
'Kid in King Arthur's Court, A ',
'Major Payne ',
'Men of the House ',
'Mimi: Rhapsody ',
'Pierce Family, The ',
'Pyromaniac's Love Story, A ',
'Roommates ',
'Rent-a-Kid ',
'Short's Sweet Six Family ',
'Speaking with Sharks ',
'National Lampoon's Senior Trip ',
'Tank Girl ',
'Tommy Boy ',
'While You Were Sleeping ',
'Live Nude Girls '])]
```

Alternative Approach Using MapReduce:

1. **Mapper:**
 - Parse the input lines and emit (year, title) for movies tagged as "Comedy" and released from 1995 onwards.
 - Handle multiple genres by splitting the genre field and emitting key-value pairs for each genre.

2. Reducer:

- Group by year and aggregate movie titles into a list.
- Output the year and corresponding movie titles.

MapReduce Pseudocode:

Mapper:

For each line in movies.csv:

Extract year, title, and genres.

If year $\geq$ 1995:

For each genre in genres:

If genre == "Comedy":

Emit (year, title)

Reducer:

For each key (year):

Aggregate titles into a list.

Emit (year, [list of titles])

Query 2: Percentage of Users Who Rated At Least One Movie as 5

Solution:

1. **Filter Ratings Dataset:** Load the ratings.csv file and filter users who rated any movie as 5.
2. **Count Unique Users:**
 - Use `distinct()` to remove duplicate user IDs and count the number of unique users who rated at least one movie as 5.
3. **Calculate Percentage:**
 - Compute the total number of unique users from the ratings dataset.

- Divide the count of users who rated a movie as 5 by the total user count.

```

//summary of data:
'Ice Cream Man ',
'The Spirit of Christmas ']]]

[14]: ratingsRDD = sc.textFile("ratings.csv")

[15]: header = ratingsRDD.first()
ratingsRDD1 = ratingsRDD.filter(lambda line: line != header)

[16]: usersGoodRates = ratingsRDD1.map(lambda x: (split_complex(x)[0], split_complex(x)[1], float(split_complex(x)[2]), split_complex(x)[3])) \
    .filter(lambda x: x[2] == 5).map(lambda x: x[0]).distinct().count()

[17]: users = ratingsRDD1.map(lambda x: (split_complex(x)[0], split_complex(x)[1], float(split_complex(x)[2]), split_complex(x)[3])) \
    .map(lambda x: x[0]).distinct().count()

[18]: print(usersGoodRates / users)

0.9393442622950819

```

Alternative Approach Using MapReduce:

1. Mapper:

- Parse the input lines and emit (userID, 1) for users who rated a movie as 5.

2. Reducer:

- Aggregate the counts to identify distinct users with a rating of 5.
- Count all distinct user IDs in the dataset to calculate the percentage.

MapReduce Pseudocode:

Mapper:

For each line in ratings.csv:

Extract userID and rating.

If rating == 5:

Emit (userID, 1)

Reducer:

For each key (userID):

Emit (userID, 1)

Driver:

Count total distinct userIDs in the dataset.

Calculate (UsersWithRating5 / TotalUsers).

Query 3: Percentage of Users with Average Rating ≥ 3.5

Solution:

1. **Calculate Average Ratings:**
 - Group ratings by user ID and compute the average rating for each user.
2. **Filter Users:**
 - Retain only users whose average rating is greater than or equal to 3.5.
3. **Calculate Percentage:**
 - Count the number of users meeting the condition and divide by the total number of users.

```
[19]: userAvgRatesRDD = ratingsRDD1.map(lambda x: (split_complex(x)[0], split_complex(x)[1], float(split_complex(x)[2]), split_complex(x)[3])) \
    .map(lambda x: (x[0], x[2])) \
    .groupByKey() \
    .map(lambda x: (x[0], sum(list(x[1])) / len(list(x[1]))))

[22]: usersGoodRates = userAvgRatesRDD.filter(lambda x: x[1] >= 3.5).map(lambda x: x[0]).distinct().count()

[23]: users = userAvgRatesRDD.map(lambda x: x[0]).distinct().count()

[24]: print(usersGoodRates / users)

0.6524590163934426
```

Alternative Approach Using MapReduce:

1. **Mapper:**

Emit (userID, rating) for each line.
2. **Reducer:**
 - Group ratings by user ID.
 - Compute the average rating for each user and emit (userID, averageRating) for users with an average rating ≥ 3.5 .
3. **Driver:**
 - Count the total number of users and users meeting the condition.
 - Compute the percentage.

MapReduce Pseudocode:

Mapper:

For each line in ratings.csv:

Extract userID and rating.

Emit (userID, rating)

Reducer:

For each key (userID):

Aggregate ratings and calculate the average.

If averageRating $\geq$ 3.5:

Emit (userID, 1)

Driver:

Count total users in the dataset.

Calculate percentage as (UsersWithAvgRating3.5 / TotalUsers).